Homework (25% of overall grade)

Introduction to programming and data visualisation

Dataset 1 (the dataset was uploaded on Blackboard: 'dataProblem1.csv')

Your company was asked to devise a machine learning model that can classify fruit based on quantitative features, such as weight and size. Your managed asked you to build a preliminary model that can classify apple vs. non-apple on a small example dataset. The dataset has about 400 entries including apples, oranges, and lemons. The columns of the dataset indicate the class of the entry (apple, orange, or lemon), the weight, width, length, and a colour-score which is an unspecified value indicating the colour of the fruit.

- Task 1 (25 points out of 100): Study the dataset. Visualise the data (e.g., using scatterplots). Try different combinations of features on the x-axis and y-axis. Different colours or markers should be used for different classes (e.g., apple vs. non-apple). The purpose of this task is to understand the dataset (e.g., is everything as expected?) and to discuss this with your manager. To do so, produce the plots and explain the meaning of the figures in plain English.
- Task 2 (30 points out of 100): Fit a binary classifier and quantify its accuracy. Visualise the classification results as we discussed in the lectures and tutorials. In addition, add a short description in plain English (e.g., by using the Python comments) to each numerical result, explaining what that value represents and what can be concluded from the analysis.
- Task 3 (10 points out of 100): Fit a 3-class classifier for apples vs. oranges vs. lemons and quantify the classification accuracy. As in task 2, please discuss the result in plain English.

Dataset 2 (link: https://osf.io/n752f/ or, on Blackboard, dataProblem2.xlsx)

Similar to the previous tasks, your company is asked to determine how accurately it is possible to identify dyslexic individuals based on a small publicly available dataset.

- Task 4 (15 points out of 100): Study the dataset. Note that you are free to search for the documentation of that data and even the related paper. Visualise the data. What appear to be the most promising features?
- Task 5 (20 points out of 100): Fit a binary classifier and quantify its accuracy. Visualise the classification results as we discussed in the lectures and tutorials. In addition, add a short description in plain English (e.g., by using the Python comments) to each numerical result, explaining what that value represents and what can be concluded from the analysis. Use the model to determine the most important features, which could be used as diagnostic markers.

Note: Remember that there may be many answers! You will be evaluated based on both your code and your understanding of the problem. Be curious! What would you tell to your manager? Can the dataset be improved? How?

```
# Importing libraries
%matplotlib inline
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
#Classifiers
from sklearn.neural_network import MLPClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.gaussian_process import GaussianProcessClassifier
from sklearn.gaussian_process.kernels import RBF
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier
from sklearn.naive_bayes import GaussianNB
# Evaluation tools
from sklearn.metrics import ConfusionMatrixDisplay
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score


# CV
from sklearn.model_selection import cross_val_score
```

```
from google.colab import drive
drive.mount('/content/drive')

dataset1 = pd.read_csv("/content/drive/MyDrive/IML_Colab/dataProblem1.csv")
```

```
Mounted at /content/drive
```

```
dataset1
```

	fruit_label	fruit_name	weight	width	height	color_score
0	1	apple	192	8.4	7.3	0.55
1	1	apple	180	8.0	6.8	0.59
2	1	apple	176	7.4	7.2	0.60
3	1	apple	178	7.1	7.8	0.92
4	1	apple	172	7.4	7.0	0.89
...	...	...	...	...	...	...
394	3	lemon	166	7.8	7.9	0.49
395	3	lemon	167	7.9	7.5	0.62
396	3	lemon	198	7.4	8.1	0.43
397	3	lemon	207	7.5	8.0	0.48
398	3	lemon	198	7.0	7.6	0.68

399 rows × 6 columns

Next steps: New interactive sheet

```
dataset1.shape
```

```
(399, 6)
```

Task 1

Task 1 (25 points out of 100): Study the dataset. Visualise the data (e.g., using scatterplots). Try different combinations of features on the x-axis and y-axis. Different colours or markers should be used for different classes (e.g., apple vs. non-apple). The purpose of this task is to understand the dataset (e.g., is everything as expected?) and to discuss this with your manager. To do so, produce the plots and explain the meaning of the figures in plain English.

```
dataset1.columns
```

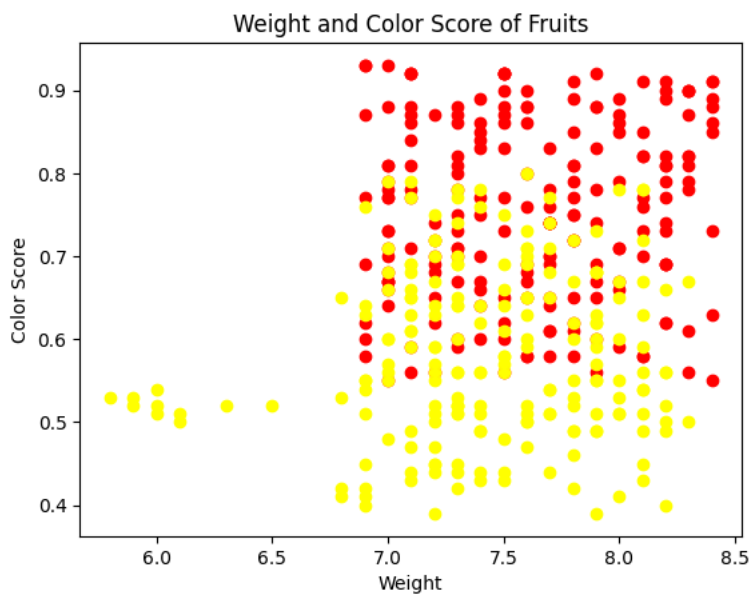
```
Index(['fruit_label', 'fruit_name', 'weight', 'width', 'height',
       'color_score'],
      dtype='object')
```

```
apple_mask = dataset1['fruit_name'] == "apple"
apples = dataset1[apple_mask]
non_apple_mask = dataset1['fruit_name'] != "apple"
non_apples = dataset1[non_apple_mask]
non_apples
```

	fruit_label	fruit_name	weight	width	height	color_score
199	2	orange	160	7.0	7.4	0.71
200	2	orange	158	7.1	7.5	0.69
201	2	orange	210	7.8	8.0	0.72
202	2	orange	164	7.2	7.0	0.70
203	2	orange	190	7.5	8.1	0.64
394	3	lemon	166	7.3	7.9	0.49
395	3	lemon	167	7.9	7.5	0.62

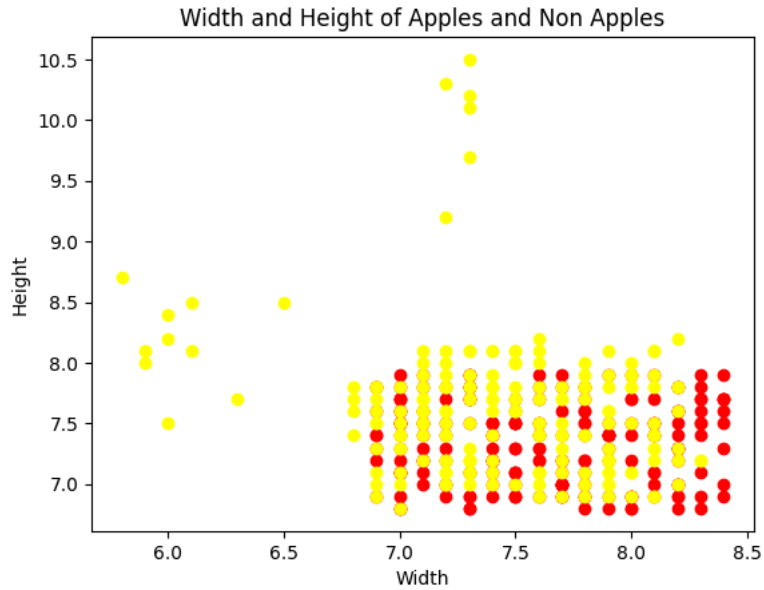
Perhaps it would be more efficent to relabel the dataset so apples have one fruit label value and non-apples have another. I need to plot these to prove my hypothesis, however there might not be great variety in the fruit features based on first glance. I would also ask to clarify the measuring scale, even though I could make a reasonable assumption, it would be best to receive total confirmation from my manager.

```
plt.scatter(apples['width'], apples['color_score'], color = 'red')
plt.scatter(non_apples['width'], non_apples['color_score'], color = 'yellow')
plt.xlabel('Weight')
plt.ylabel('Color Score')
plt.title('Weight and Color Score of Fruits')
plt.show()
```



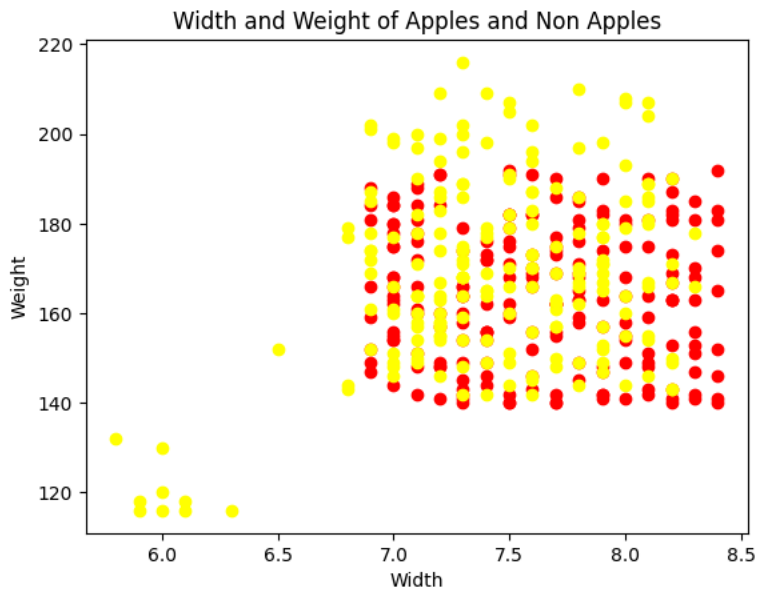
This graph reveals that some apples have much higher color scores than non-apples, there is a great deal of overlap, from looking at the plot, in the weight.

```
plt.scatter(apples['width'], apples['height'], color = 'red')
plt.scatter(non_apples['width'], non_apples['height'], color = 'yellow')
plt.xlabel('Width')
plt.ylabel('Height')
plt.title('Width and Height of Apples and Non Apples')
plt.show()
```



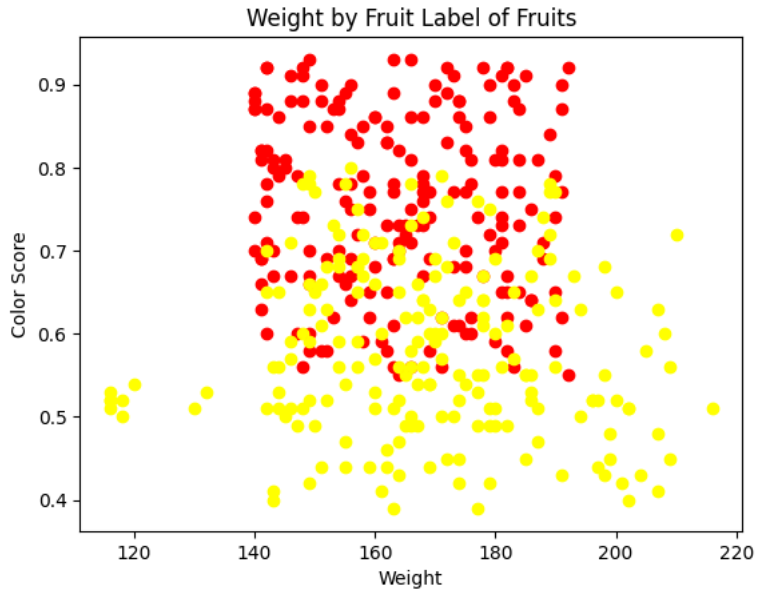
When plotting height and Weight, without the graph coloring, apples would almost be indistinguishable from the non-apples, indicating that the fruits in the dataset have similar heights around 7-8 and widths around 7-8. However, it is important to note that a sizeable number of apples have a width greater than non-apples when approaching 8.5.

```
plt.scatter(apples['width'], apples['weight'], color = 'red')
plt.scatter(non_apples['width'], non_apples['weight'], color = 'yellow')
plt.xlabel('Width')
plt.ylabel('Weight')
plt.title('Width and Weight of Apples and Non Apples')
plt.show()
```



The weight of apples and non-apples is pretty much identical between 140 and 185, with some distinguishable non-apples being around 200 and around 120-140. In addition, as mentioned above, some apples are wider than non-apples, about 8.3. Outside of those points, the data overlap quite a bit, which may affect the classification algorithm's accuracy, especially so with a multiclass model.

```
plt.scatter(apples['weight'], apples['color_score'], color = 'red')
plt.scatter(non_apples['weight'], non_apples['color_score'], color = 'yellow')
plt.xlabel('Weight')
plt.ylabel('Color Score')
plt.title('Weight by Fruit Label of Fruits')
plt.show()
```



These plots reveal a possible impact on the algorithm's accuracy. When plotting various features, i.e., width, weight, and height, there is a great deal of overlap between apples and non-apples. The main feature with greater ability to distinguish apples from non-apples is the color score, as seen above. When plotting width and height, there was no way to distinguish apples from non-apples, indicating that apples and the non-apples share similar sizes. Further, we can see there is little variation in apple values compared to non-apple values. Therefore, it would be recommended to incorporate a wider variety of fruits for classification, for example, melons and blueberries, both of which vary greatly in size and weight. If not incorporating different fruits, perhaps finding other scales of measurement to differentiate. Beyond a binary classification model, a multiclass model might suffer more, given that there is some differentiation between apples and non-apples, but even greater overall with oranges and lemons. However, determining variability solely from visual inspection is difficult, so to better understand the difference between apples and non-apples, we can group them and compute the mean of the various features.

```
total = (dataset1.groupby( #grouping fruits and finding mean weight
    ['fruit_name'], observed=True
)['weight'].mean().reset_index())

total_sorted = total.sort_values(by='weight', ascending=False)
print(total_sorted.reset_index(drop=True))
```

```
  fruit_name      weight
0      lemon  171.860000
1     orange  165.640000
2      apple  164.170854
```

```
total = (dataset1.groupby( #grouping fruits and finding mean height
    ['fruit_name'], observed=True
)['height'].mean().reset_index())

total_sorted = total.sort_values(by='height', ascending=False)
print(total_sorted.reset_index(drop=True))
```

```
  fruit_name    height
0      lemon  7.849000
1     orange  7.386000
2      apple  7.347236
```

```
total = (dataset1.groupby( #grouping fruits and finding mean width
    ['fruit_name'], observed=True
)['width'].mean().reset_index())

total_sorted = total.sort_values(by='width', ascending=False)
print(total_sorted.reset_index(drop=True))
```

```
  fruit_name     width
0      apple  7.626131
1     orange  7.509000
2      lemon  7.346000
```

```
total = (dataset1.groupby( #grouping fruits and finding mean color_score
    ['fruit_name'], observed=True
)['color_score'].mean().reset_index())

total_sorted = total.sort_values(by='color_score', ascending=False)
print(total_sorted.reset_index(drop=True))
```

```
  fruit_name  color_score
0      apple     0.755427
1     orange     0.641200
2      lemon     0.519100
```

Looking at the means of the various features grouped by the different fruits, it becomes obvious that there is great overlap between the fruits in every category besides color score which has the greatest variation. As stated earlier, this may impact the accuracy of the anticipated algorithm as there will be great class overlap.

Task 2

```
import numpy as np

#creating X and Y
X = dataset1.drop(columns=['fruit_label', 'fruit_name'])
y = dataset1['fruit_label']


#Randomised Split
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2,
                                                    random_state=42, stratify=y)
print(np.bincount(y_test))

#setting the binary classification where 1 = apples
y_train_1 = (y_train == 1)
y_test_1 = (y_test == 1)
from sklearn.svm import SVC
clf = SVC(kernel = 'linear')
clf.fit(X_train, y_train_1)
```

```
[ 0 40 20 20]
▾ SVC
SVC(kernel='linear')
```

```
#Here we are predicting new values based on the train and test dataset
predicted_y_train = clf.predict(X_train)
predicted_y_test = clf.predict(X_test)
```

```
#showing us how well the train corresponds to real values
print("Number of mislabeled points in the training set out of a total %d points: %d" %
      (X_train.shape[0], (y_train_1 != predicted_y_train).sum()))
print("Number of mislabeled points in the testing set out of a total %d points: %d" %
      (X_test.shape[0], (y_test_1 != predicted_y_test).sum()))
```

```
Number of mislabeled points in the training set out of a total 319 points: 73
Number of mislabeled points in the testing set out of a total 80 points: 13
```

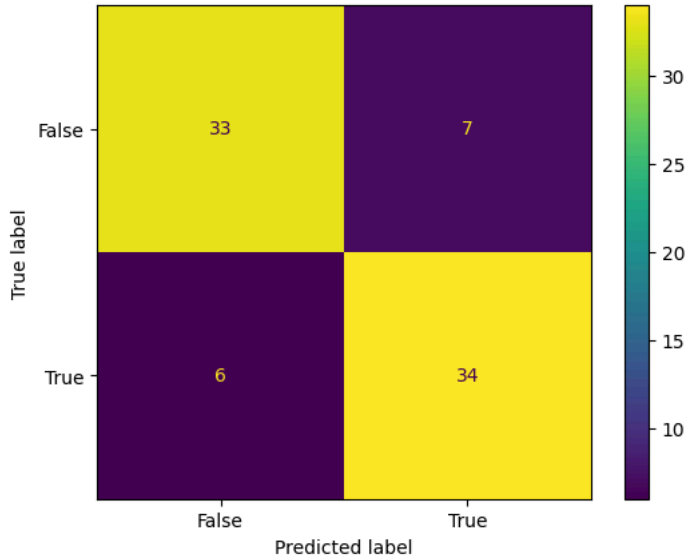
```
test_score = clf.score(X_test, y_test_1)
train_score = clf.score(X_train, y_train_1)
print('Score', train_score, test_score)
```

```
Score 0.7711598746081505 0.8375
```

The model performs slightly better on the test set than on the training set. This is an indication of that our model is not overfitting.

```
ConfusionMatrixDisplay.from_estimator(clf, X_test, y_test_1)
```

```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x7981e1806180>
```




```
accuracy = accuracy_score(y_test_1, predicted_y_test)
recall = recall_score(y_test_1, predicted_y_test, average = "weighted")
precision = precision_score(y_test_1, predicted_y_test, average = "weighted")
print("Accuracy Score:", accuracy)
print("Recall Score:", recall)
print("Precision Score:", precision)
```

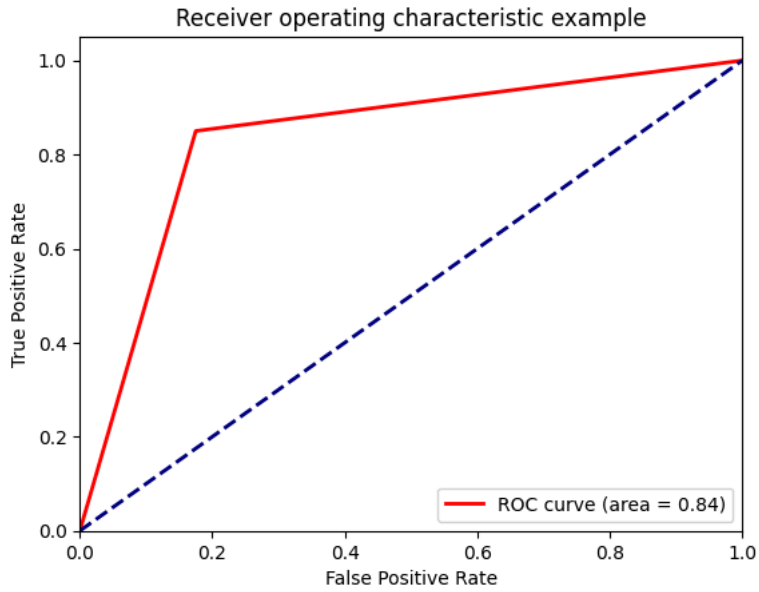
```
Accuracy Score: 0.8375
Recall Score: 0.8375
Precision Score: 0.8377110694183866
```

This chart shows us the number of correct and incorrect labels the algorithm has. The two yellow boxes, with the higher values indicate correct labels, representing true positive and true negative. Having higher values in the yellow diagonal indicates that our model's predictions relatively match the dataset and is an indicator of strong predictive capability. The purple diagonal indicates false negatives and false positives. We can use this to determine accuracy and F1 score, which measures a balance of precision and recall.

```
accuracy = accuracy_score(y_test_1, predicted_y_test)
f1 = f1_score(y_test_1, predicted_y_test)
print("Accuracy Score:", accuracy)
print("F1 Score:", f1)
```

```
Accuracy Score: 0.8375
F1 Score: 0.8395061728395061
```

```
from sklearn.metrics import roc_curve, auc

fpr, tpr, thresholds = roc_curve(y_test_1, predicted_y_test)
roc_auc = auc(fpr, tpr)

plt.figure()
plt.plot(fpr, tpr, color='red', lw=2, label='ROC curve (area = %0.2f)' % roc_auc)
plt.plot([0, 1], [0, 1], color='navy',lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver operating characteristic example')
plt.legend(loc='lower right')
plt.show()
```




```
import numpy as np
#creating X and Y
X1 = dataset1.drop(columns=['fruit_label', 'fruit_name'])
y = dataset1['fruit_label']
#Randomised Split
from sklearn.model_selection import train_test_split
X1_train, X1_test, y_train, y_test = train_test_split(X1, y, test_size = 0.2,
                                                      random_state=42, stratify=y)
print(np.bincount(y_test))
## training
y_train_1x = (y_train == 1)
y_test_1x = (y_test == 1)
from sklearn import tree
clf = tree.DecisionTreeClassifier(max_depth=100)
clf.fit(X1_train, y_train_1x)
```

```
[ 0 40 20 20]
▾ DecisionTreeClassifier
DecisionTreeClassifier(max_depth=100)
```

```
predicted_y_trainx = clf.predict(X1_train)
predicted_y_testx = clf.predict(X1_test)
```

Double-click (or enter) to edit

```
print("Number of mislabeled points in the training set out of a total %d points: %d"
      % (X1_train.shape[0], (y_train_1x != predicted_y_trainx).sum()))
print("Number of mislabeled points in the testing set out of a total %d points: %d"
 % (X1_test.shape[0], (y_test_1x != predicted_y_testx).sum()))
```

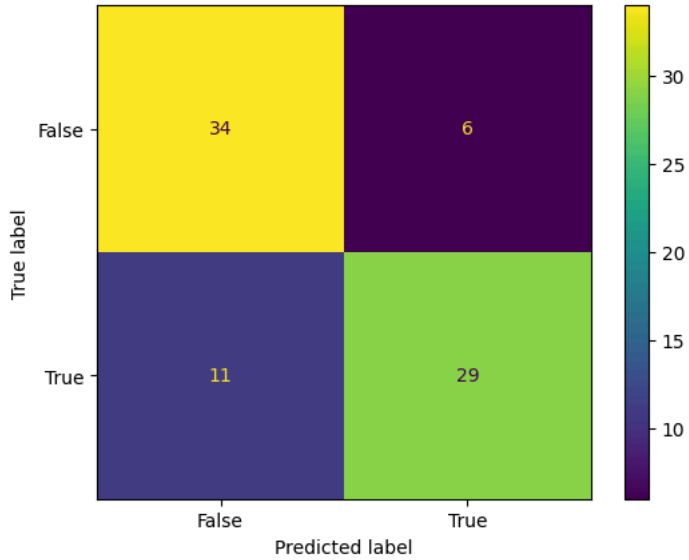
```
Number of mislabeled points in the training set out of a total 319 points: 0
Number of mislabeled points in the testing set out of a total 80 points: 17
```

```
test_score = clf.score(X1_test, y_test_1x)
train_score = clf.score(X1_train, y_train_1x)
print('Score', train_score, test_score)
```

```
Score 1.0 0.7875
```

```
ConfusionMatrixDisplay.from_estimator(clf, X1_test, y_test_1x)
```

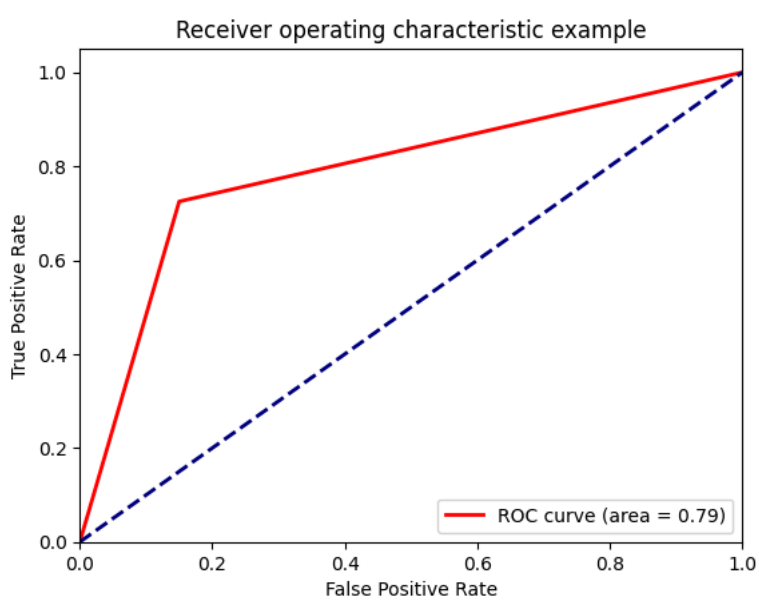
```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x7981e0942240>
```




```
accuracy = accuracy_score(y_test_1x, predicted_y_testx)
f1 = f1_score(y_test_1x, predicted_y_testx)
print("Accuracy Score:", accuracy)
print("F1 Score:", f1)
```

```
Accuracy Score: 0.7875
F1 Score: 0.7733333333333333
```

```
from sklearn.metrics import roc_curve, auc

fpr, tpr, thresholds = roc_curve(y_test_1x, predicted_y_testx)
roc_auc = auc(fpr, tpr)

plt.figure()
plt.plot(fpr, tpr, color='red', lw=2, label='ROC curve (area = %0.2f)' % roc_auc)
plt.plot([0, 1], [0, 1], color='navy',lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver operating characteristic example')
plt.legend(loc='lower right')
plt.show()
```



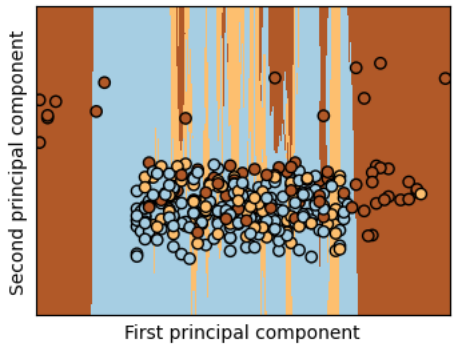
In order to establish a baseline and determine the strength of a mdoel, it is crucial to compare it to different classifiers. When using the second classfier, the model preformed worse than the first. Therefore the first model is ideal. It has an F1 score of about 83% and an accuracy score of 85%. This means the model correctly predicts 85% of the cases. Further an F1 score of 83% means there is a strong balances of precision, in which when the model predicts of positive case it is correct about 83 percent of the time and it includes recall which shows us how much of all positive casts the model detects. Further, the classifier preformed relatively well with an Area Under the Curve (AUC) of 0.85, which indicates a high capability to distinguish between apples and non-apples. Therefore I reccomend using the first model, given its higher accuracy, AUC, and F1 score.

Task 3

```
from sklearn.neighbors import KNeighborsClassifier
import numpy as np
X = dataset1.drop(columns=['fruit_label', 'fruit_name'])
y = dataset1['fruit_label']
#Randomised Split
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2,
                                                    random_state=42)
## training
knn =  KNeighborsClassifier(8)
knn.fit(X_train, y_train)
```

```
▾ KNeighborsClassifier
KNeighborsClassifier(n_neighbors=8)
```

```
from sklearn.decomposition import PCA

# Reduce the dimensionality of the data to 2 principal components
pca = PCA(n_components=2)
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)

# Fit the classifier on the reduced data
knn.fit(X_train_pca, y_train)

# Create a meshgrid to plot the decision boundary
h = .02  # step size in the mesh
x_min, x_max = X_train_pca[:, 0].min() - 1, X_train_pca[:, 0].max() + 1
y_min, y_max = X_train_pca[:, 1].min() - 1, X_train_pca[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))

# Predict the class for each point in the meshgrid
Z = knn.predict(np.c_[xx.ravel(), yy.ravel()])

# Put the result into a color plot
Z = Z.reshape(xx.shape)
plt.figure(1, figsize=(4, 3))
plt.pcolormesh(xx, yy, Z, cmap=plt.cm.Paired)

# Plot also the training points
plt.scatter(X_train_pca[:, 0], X_train_pca[:, 1], c=y_train, edgecolors='k',
            cmap=plt.cm.Paired)
plt.xlabel('First principal component')
plt.ylabel('Second principal component')

plt.xlim(xx.min(), xx.max())
plt.ylim(yy.min(), yy.max())
plt.xticks(())
plt.yticks(())

plt.show()
```




```
predicted_y_train = knn.predict(X_train)
predicted_y_test = knn.predict(X_test)
predicted_y_test.shape
```

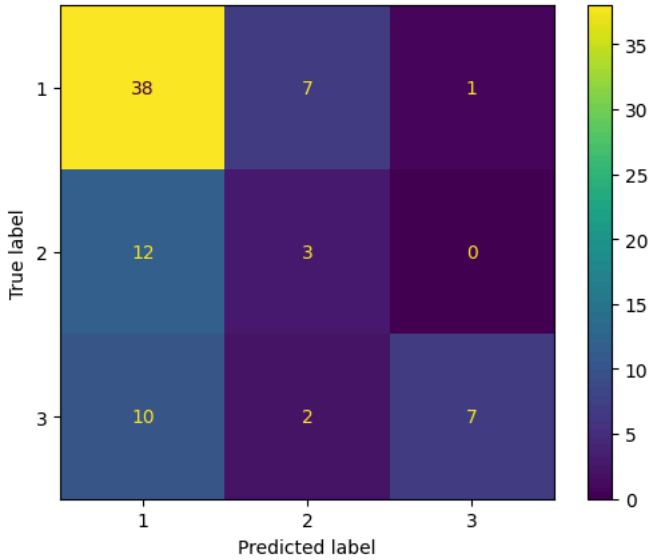
```
(80,)
```

```
print("Number of mislabeled points in the training set out of a total %d points : %d"
 % (X_train.shape[0], (y_train != predicted_y_train).sum()))

print("Number of mislabeled points in the testing set out of a total %d points : %d"
 % (X_test.shape[0], (y_test != predicted_y_test).sum()))
test_score = knn.score(X_test, y_test)
train_score = knn.score(X_train, y_train)
print('Score', train_score, test_score)
```

```
Number of mislabeled points in the training set out of a total 319 points : 113
Number of mislabeled points in the testing set out of a total 80 points : 32
Score 0.64576802507837 0.6
```

```
ConfusionMatrixDisplay.from_estimator(knn, X_test, y_test)
```

```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x7981e184f8f0>
```

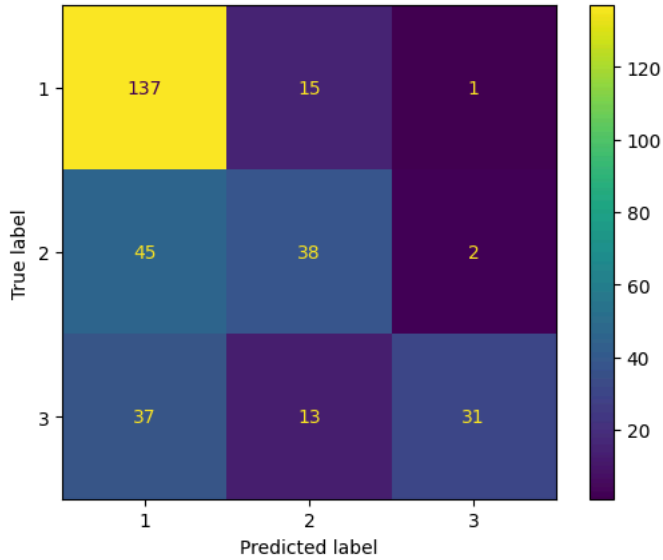



```
accuracy = accuracy_score(y_test, predicted_y_test)
recall = recall_score(y_test, predicted_y_test, average = "weighted")
precision = precision_score(y_test, predicted_y_test, average = "weighted")
print("Accuracy Score:", accuracy)
print("Recall Score:", recall)
print("Precision Score:", precision)
```

```
Accuracy Score: 0.6
Recall Score: 0.6
Precision Score: 0.6188541666666667
```

```
ConfusionMatrixDisplay.from_estimator(knn, X_train, y_train)
```

```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x7981e1807110>
```

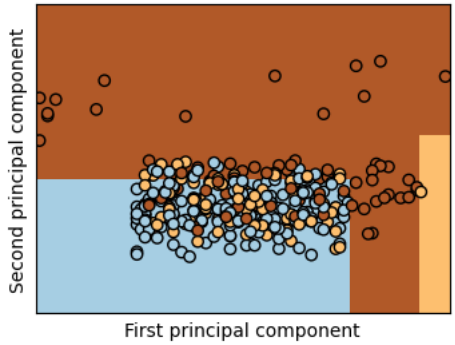



```
from sklearn import tree
import numpy as np
X = dataset1.drop(columns=['fruit_label', 'fruit_name'])
y = dataset1['fruit_label']
#Randomised Split
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2,
                                                    random_state=42)
## training
tree =  DecisionTreeClassifier(max_depth=3)
tree.fit(X_train, y_train)
```

```
DecisionTreeClassifier
DecisionTreeClassifier(max_depth=3)
```

```
from sklearn.decomposition import PCA

# Reduce the dimensionality of the data to 2 principal components
pca = PCA(n_components=2)
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)

# Fit the classifier on the reduced data
tree.fit(X_train_pca, y_train)

# Create a meshgrid to plot the decision boundary
h = .02  # step size in the mesh
x_min, x_max = X_train_pca[:, 0].min() - 1, X_train_pca[:, 0].max() + 1
y_min, y_max = X_train_pca[:, 1].min() - 1, X_train_pca[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))

# Predict the class for each point in the meshgrid
Z = tree.predict(np.c_[xx.ravel(), yy.ravel()])

# Put the result into a color plot
Z = Z.reshape(xx.shape)
plt.figure(1, figsize=(4, 3))
plt.pcolormesh(xx, yy, Z, cmap=plt.cm.Paired)

# Plot also the training points
plt.scatter(X_train_pca[:, 0], X_train_pca[:, 1], c=y_train, edgecolors='k',
            cmap=plt.cm.Paired)
plt.xlabel('First principal component')
plt.ylabel('Second principal component')

plt.xlim(xx.min(), xx.max())
plt.ylim(yy.min(), yy.max())
plt.xticks(())
plt.yticks(())
```

```
plt.show()
```



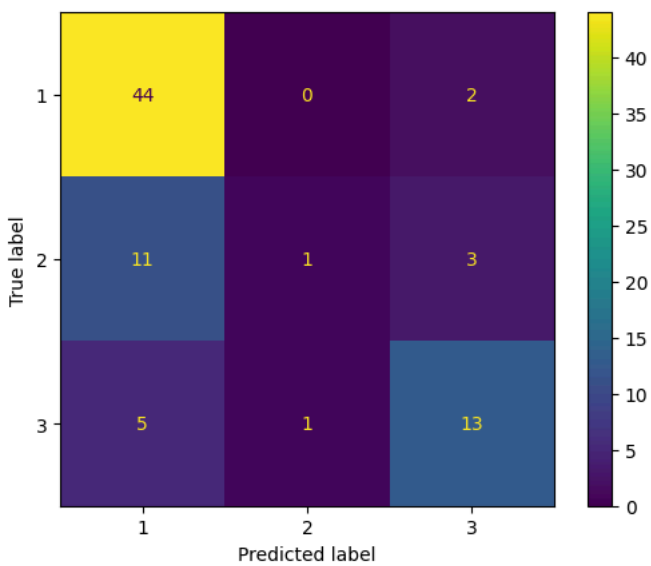

```
pred_y_tree = tree.predict(X_train)
pred_ys = tree.predict(X_test)
```

```
print("Number of mislabeled points in the training set out of a total %d points : %d" % (X_train.shape[0], (y_train

print("Number of mislabeled points in the testing set out of a total %d points : %d" % (X_test.shape[0], (y_test !=
test_score = tree.score(X_test, y_test)
train_score = tree.score(X_train, y_train)
print('Score', test_score, train_score)
```

```
Number of mislabeled points in the training set out of a total 319 points : 90
Number of mislabeled points in the testing set out of a total 80 points : 22
Score 0.725 0.7178683385579937
```

```
ConfusionMatrixDisplay.from_estimator(tree, X_test, y_test)
```

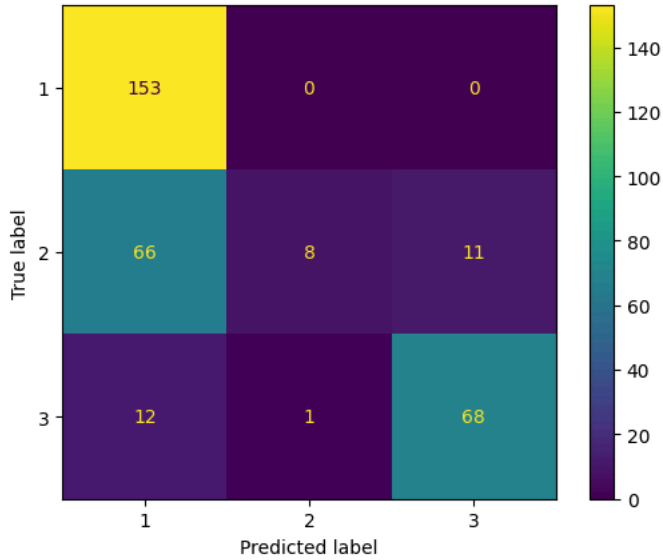
```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x7981e08d2c60>
```




```
accuracy = accuracy_score(y_test, pred_ys)
recall = recall_score(y_test, pred_ys, average = "weighted")
precision = precision_score(y_test, pred_ys, average = "weighted")
print("Accuracy Score:", accuracy)
print("Recall Score:", recall)
print("Precision Score:", precision)
```

```
ConfusionMatrixDisplay.from_estimator(tree, X_train, y_train)
```

```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x7981e18625a0>
```



For this classifier, given the overlap in fruit features, the accuracy of the multi-class models may be affected. We can see this in the PCA plot, which also shows the decision boundaries. From the plots in both models, particularly in the first, there is significant overlap between the boundaries, indicating that the data itself has substantial overlap and that the models struggle to properly separate the different classes. However, the boundaries in the second model are much clearer, and they are clearly defined. This fact is corroborated in two different places. First, the overlap is evident when plotting the features and referencing the means. The second place this is covered is in the confusion matrix. The confusion matrix indicates that the model is better at predicting apples than the other two fruits; however, there is significant overlap when classifying lemons and oranges. Generally, I recommend utilizing the second model over the first. The first model shows a slight case of overfitting, as evidenced by its higher accuracy on the training set than on the test set. In comparison, the second model achieved 72% accuracy on the test set and 71% on the training set, indicating the model performed better on real data. Second, the confusion matrix for the second model shows it has higher overall predictive capability, as indicated by the top-left-to-bottom-right diagonal in all three matrices. At the same time, the first model's predictive capabilities are significantly worse. The final metrics used to determine the accuracy of the model are accuracy, recall, and precision scores. The second model performed better on this metric as well, with an accuracy score of 72%, a precision score of about 72%, and a recall score of about 68%. Overall, there is a fundamental problem within the dataset itself. If you want to improve the model's accuracy, I recommend adding more physical features to better distinguish fruits with greater differences. As we saw in the earlier plot, there is significant overlap among the fruit values.

Task 4

```
from google.colab import drive
drive.mount('/content/drive')

dataset2 = pd.read_excel("/content/drive/MyDrive/IML_Colab/dataProblem2.xlsx")
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_rem
```

```
print(dataset2)
```

```
   Dyslexia  TTO  TRO  TNO  RTO  RRO  RNO  NTO  NRO  NNO  TTS  TRS  TNS  RTS  \
0         y   13    3    8    2   15    7    6    6   12    5   12    7    7
1         y   16    4    4    1   12   11    0   14   10   15    7    2    2
2         y   15    4    5    3    8   13    4    1   19   12    2   10    7
3         y   12    6    6    2   11   11    0    6   18   14    5    5    1
4         y   17    4    3    4    9   11    3    5   16   18    2    4    2
..      ...  ...  ...  ...  ...  ...  ...  ...  ...  ...  ...  ...  ...  ...
66        n   15    4    5    1   10   13    4    2   18   15    4    5    0
67        n   21    1    2    4    9   11    5    3   16   22    1    1    2
68        n   19    3    2    8   11    5    3    6   15   15    6    3    7
69        n   15    0    9    4    5   15    2    0   22   18    1    5    0
70        n   22    0    2    3   11   10    0    3   21   10    7    7    3

    RRS  RNS  NTS  NRS  NNS
0    11    6    8   12    4
1    14    8    1    8   15
2     9    8    0    4   20
3    12   11    1    7   16
```

```
4    15    7    2    9   13
..  ...  ...  ...  ...  ...
66   15    9    0    3   21
67   14    8    1    8   15
68   12    5   17    3    4
69   13   11    0    4   20
70    8   13    0    8   16

[71 rows x 19 columns]
```

```
dataset2.dtypes
```

	0
Dyslexia	object
TTO	int64
TRO	int64
TNO	int64
RTO	int64
RRO	int64
RNO	int64
NTO	int64
NRO	int64
NNO	int64
TTS	int64
TRS	int64
TNS	int64
RTS	int64
RRS	int64
RNS	int64
NTS	int64
NRS	int64
NNS	int64

dtype: object

```
mask = dataset2['Dyslexia'] == 'y'
dataset2.loc[mask, 'Dyslexia'] = 1
dataset2.loc[~mask, 'Dyslexia'] = 0
```

I am making the personal choice to onvert the y and n into 1 and 0 for clarity when running the models.

```
dataset2
```

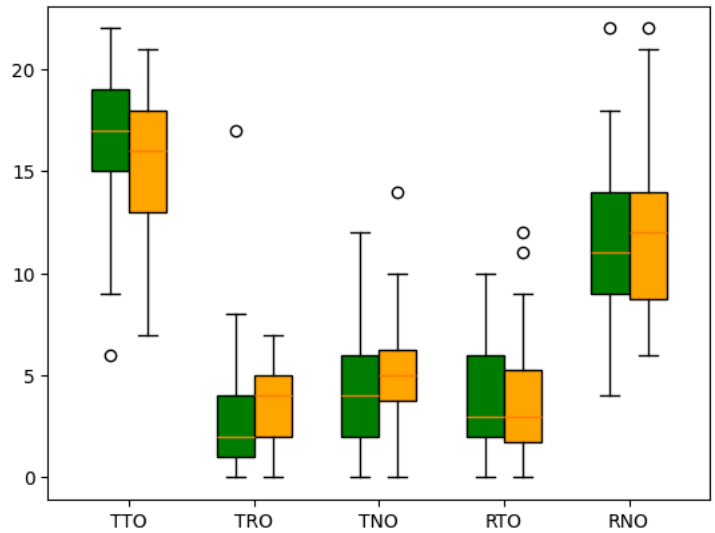
	Dyslexia	TTO	TRO	TNO	RTO	RRO	RNO	NTO	NRO	NNO	TTS	TRS	TNS	RTS	RRS	RNS	NTS	NRS	NNS
0	1	13	3	8	2	15	7	6	6	12	5	12	7	7	11	6	8	12	4
1	1	16	4	4	1	12	11	0	14	10	15	7	2	2	14	8	1	8	15
2	1	15	4	5	3	8	13	4	1	19	12	2	10	7	9	8	0	4	20
3	1	12	6	6	2	11	11	0	6	18	14	5	5	1	12	11	1	7	16
4	1	17	4	3	4	9	11	3	5	16	18	2	4	2	15	7	2	9	13
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
66	0	15	4	5	1	10	13	4	2	18	15	4	5	0	15	9	0	3	21
67	0	21	1	2	4	9	11	5	3	16	22	1	1	2	14	8	1	8	15
68	0	19	3	2	8	11	5	3	6	15	15	6	3	7	12	5	17	3	4
69	0	15	0	9	4	5	15	2	0	22	18	1	5	0	13	11	0	4	20
70	0	22	0	2	3	11	10	0	3	21	10	7	7	3	8	13	0	8	16

71 rows × 19 columns

Next steps: New interactive sheet

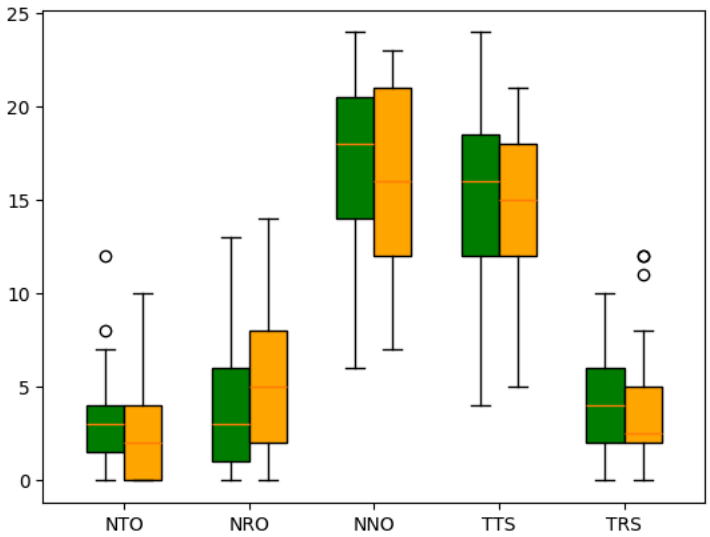
```
dyslexicm = dataset2['Dyslexia'] == 0
dyslexicn = dataset2['Dyslexia'] == 1
```

```
x1 = ['TTO', 'TRO', 'TNO', 'RTO', 'RNO']
dys_data = [dataset2.loc[dyslexicm, col] for col in x1]
non_data = [dataset2.loc[dyslexicn, col] for col in x1]

width = 0.30
x = np.arange(len(x1))

plt.figure()
plt.boxplot(dys_data, positions=x - width/2, widths=width, patch_artist=True,
            boxprops=dict(facecolor='green'))
plt.boxplot(non_data, positions=x + width/2, widths=width, patch_artist=True,
            boxprops=dict(facecolor='orange'))
plt.xticks(x, x1)
plt.show()
```

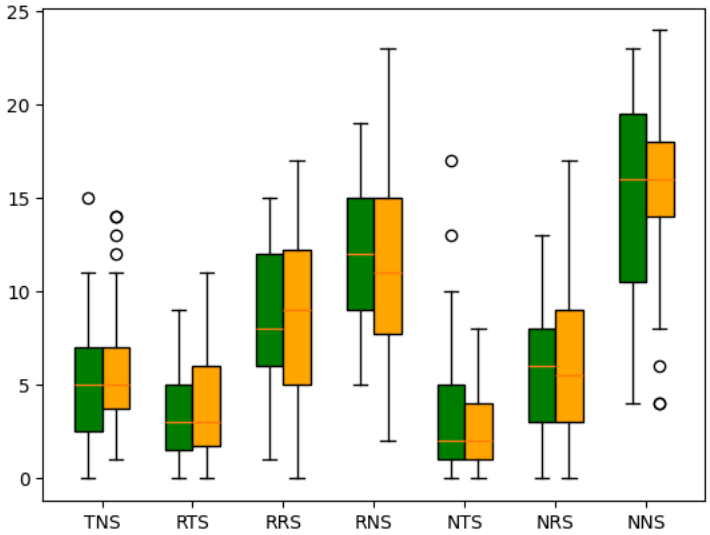



```
x2 = ['NTO', 'NRO', 'NNO', 'TTS', 'TRS']
dys_data = [dataset2.loc[dyslexicm, col] for col in x2]
non_data = [dataset2.loc[dyslexicn, col] for col in x2]

width = 0.30
x_ = np.arange(len(x2))

plt.figure()
```

```
plt.boxplot(dys_data, positions=x_ - width/2, widths=width, patch_artist=True,
            boxprops=dict(facecolor='green'))
plt.boxplot(non_data, positions=x_ + width/2, widths=width, patch_artist=True,
            boxprops=dict(facecolor='orange'))
plt.xticks(x_, x2)
plt.show()
```




```
x3 = ['TNS', 'RTS', 'RRS', 'RNS', 'NTS', "NRS", 'NNS']
dys_data = [dataset2.loc[dyslexicm, col] for col in x3]
non_data = [dataset2.loc[dyslexicn, col] for col in x3]

width = 0.30
xx = np.arange(len(x3))

plt.figure()
plt.boxplot(dys_data, positions=xx - width/2, widths=width, patch_artist=True,
            boxprops=dict(facecolor='green'))
plt.boxplot(non_data, positions=xx + width/2, widths=width, patch_artist=True,
            boxprops=dict(facecolor='orange'))
plt.xticks(xx, x3)
plt.show()
```



To visualize the features that may be most important in determining whether someone has dyslexia, I used a box plot to plot the means. As mentioned above, ensuring a difference in means for the classes is an important factor in model accuracy. Therefore, some features with significant variation are NRO, NTS, and NRS. Both of which are tests of orthographical and semantic conditions. I also noticed there is a bit of redundancy in the dataset when comparing the graphs to their descriptions. The redundancy comes from looking at the means between the number of answers 'related' and 'new.' In a few of the various conditions, the results for those two responses are generally indicative of Dyslexia. However, this may not be an issue and can strengthen the dataset; it is a noticeable redundancy.

```
total = (
    dataset2
    .groupby('Dyslexia', observed=True)
    .mean(numeric_only=True)
    .reset_index()
)

total_sorted = total.sort_values(by='Dyslexia', ascending=False)
print(total_sorted.reset_index(drop=True))
```

```
   Dyslexia        TTO       TRO       TNO       RTO      RRO        RNO  \
0         1  15.468750  3.531250  5.000000  3.781250  7.84375  12.375000
1         0  16.717949  2.717949  4.564103  3.974359  8.74359  11.282051

        NTO       NRO        NNO        TTS       TRS       TNS       RTS  \
0  2.593750  5.250000  16.156250  14.375000  3.906250  5.718750  3.843750
1  3.205128  3.974359  16.820513  15.076923  3.974359  4.948718  3.487179

        RRS        RNS       NTS       NRS        NNS
0  8.781250  11.375000  2.656250  6.062500  15.281250
1  8.435897  12.076923  3.512821  5.564103  14.923077
```

To reaffirm my reading of the graphs, I also calculated the means of each feature to theur resecptive category and found the features highlighted above are possibly some of the most promising. Based on initial visual inspection as well as looking simply at the means.

```
X = dataset2.drop(columns=['Dyslexia'])
```

```
y = dataset2['Dyslexia']
```

Task 5

```
import numpy as np
#Randomised Split
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn import preprocessing

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2,
                                                    random_state=42, stratify=y)
print(np.bincount(y_test))
## training
y_train_y = (y_train == 1)
y_test_y = (y_test == 1)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.fit_transform(X_test)
from sklearn.svm import SVC
from sklearn import svm
clf = svm.SVC(kernel = 'linear', gamma=0.01, C=90)
clf.fit(X_train, y_train_y)
```

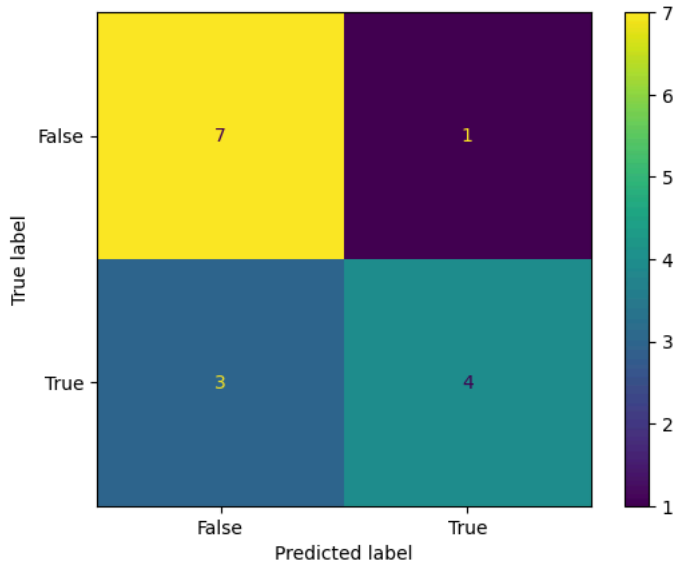
```
[8 7]
▾ SVC
SVC(C=90, gamma=0.01, kernel='linear')
```

```
predicted_y_train = clf.predict(X_train)
predicted_y_test = clf.predict(X_test)

#How well does it correspond to the 'real' values?
print("Number of mislabeled points in the training set out of a total %d points : %d"
      % (X_train.shape[0], (y_train != predicted_y_train).sum()))
print("Number of mislabeled points in the testing set out of a total %d points : %d"
      % (X_test.shape[0], (y_test != predicted_y_test).sum()))

#Score: Correctly classified over total number of points
train_score = clf.score(X_train, y_train_y)
test_score = clf.score(X_test, y_test_y)

print('Score: ', train_score, test_score)
```

```
Number of mislabeled points in the training set out of a total 56 points : 13
Number of mislabeled points in the testing set out of a total 15 points : 4
Score:  0.7678571428571429 0.7333333333333333
```

```
ConfusionMatrixDisplay.from_estimator(clf, X_test, y_test_y)
```

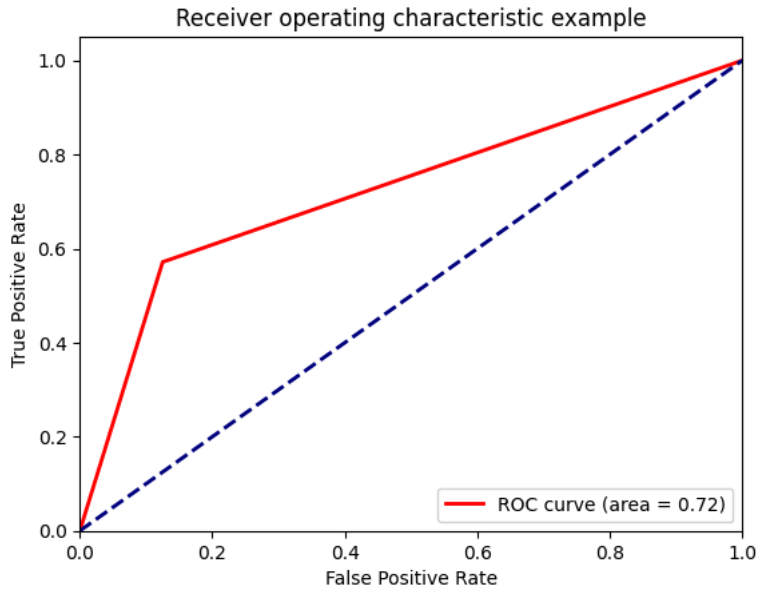
```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x7981d99aa930>
```




```
accuracy = accuracy_score(y_test_y, predicted_y_test)
recall = recall_score(y_test_y, predicted_y_test, average = "weighted")
precision = precision_score(y_test_y, predicted_y_test, average = "weighted")
print("Accuracy Score:", accuracy)
print("Recall Score:", recall)
print("Precision Score:", precision)
```

```
Accuracy Score: 0.7333333333333333
Recall Score: 0.7333333333333333
Precision Score: 0.7466666666666666
```

```
from sklearn.metrics import roc_curve, auc

fpr, tpr, thresholds = roc_curve(y_test_y, predicted_y_test)
roc_auc = auc(fpr, tpr)

plt.figure()
plt.plot(fpr, tpr, color='red', lw=2, label='ROC curve (area = %0.2f)' % roc_auc)
plt.plot([0, 1], [0, 1], color='navy',lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver operating characteristic example')
plt.legend(loc='lower right')
plt.show()
```




```
import numpy as np
#Randomised Split
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2,
                                                    random_state=42, stratify=y)
print(np.bincount(y_test))
## training
y_train = y_train.astype(int)
y_test = y_test.astype(int)
from sklearn import svm
#here we are fitting our classifier
clf = KNeighborsClassifier(5)
clf.fit(X_train, y_train_y)
```

```
[8 7]
▾ KNeighborsClassifier
KNeighborsClassifier()
```

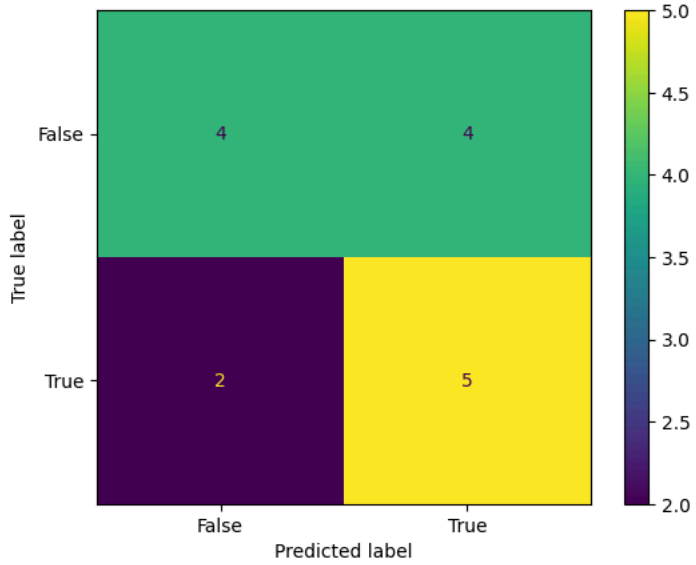
```
#predicting new values based on the train dataset and the test dataset
pred_y_train = clf.predict(X_train)
pred_y_test = clf.predict(X_test)

#seeing how well the training set preforms and corresponds compared to real values
print("Number of mislabeled points in the training set out of a total %d points: %d"
 % (X_train.shape[0], (y_train != pred_y_train).sum()))
print("Number of mislabeled points in the testing set out of a total %d points: %d"
 % (X_test.shape[0], (y_test != pred_y_test).sum()))

test_score = clf.score(X_test, y_test_y)
train_score = clf.score(X_train, y_train_y)

print('Score', train_score, test_score)
```

```
Number of mislabeled points in the training set out of a total 56 points: 15
Number of mislabeled points in the testing set out of a total 15 points: 6
Score 0.7321428571428571 0.6
```

```
accuracy = accuracy_score(y_test, pred_y_test)
recall = recall_score(y_test, pred_y_test, average = "weighted")
precision = precision_score(y_test, pred_y_test, average = "weighted")
print("Accuracy Score:", accuracy)
print("Recall Score:", recall)
print("Precision Score:", precision)
```

```
Accuracy Score: 0.6
Recall Score: 0.6
Precision Score: 0.6148148148148148
```

```
ConfusionMatrixDisplay.from_estimator(clf, X_test, y_test_y)
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x7981e0564ce0>
```




```
from sklearn.metrics import roc_curve, auc

fpr, tpr, thresholds = roc_curve(y_test_y, pred_y_test)
roc_auc = auc(fpr, tpr)

plt.figure()
plt.plot(fpr, tpr, color='red', lw=2, label='ROC curve (area = %0.2f)' % roc_auc)
plt.plot([0, 1], [0, 1], color='navy',lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
```